Assignment-2

① Reverse a string using stack.

Algorithm:-

① Push each character of the string onto the stack.

② Pop character one by one and store them back into the string.

③ Print the reversed string.

Code:-

```
#include <stdio.h>
#include <string.h>

# define MAX 100

// Stack structure
char stack[MAX];
int top = -1;

// Push character onto stack.
void push (char c){
    stack [++top] = c;
}

//Pop character from stack.
char pop(){
    return stack [top--];
}
```

```
int main() {
    char str[MAX];
printf("Enter a string");
scanf("%s", str);

int len = strlen(str);

// push all characters onto stack.
  for (int i=0 ; i<len ; i++) {
    push(str[i]); }

// pop character to reverse
  for (int i=0 ; i<len ; i++) {
      str[i] = pop();

printf("Reversed string: %s \n", str);
  return 0;
  }
```

output:-

Enter a string : DATA

Reversed string : ATAD

② Balanced Parentheses:-

Algorithm:-

① Scan each character of the expresion.

② if it's (, Push it onto the stack.

③ if it's), check if stack is empty - if yes, unbalenced; else Pop.

④ After scanning, if stack is empty → Balanced, else ⇒ unbalenced.

Code:-

```
#include<stdio.h>
#include<string.h>
# define MAX 100

char stack [MAX];

int top=-1;

//Push onto stack.

void Push (char c){

    stack [++top] = c;

}

//pop from stack.

char Pop (){

    return stack [top--];

}
```

```
int main(){
  char expr[MAX];
  int balanced= 1;
printf("Enter expression: ");
int len =strlen(expr);
for (int i=0 ; i<len;i++){
  if (expr[i] ==='c'){
    Push (expr[i]); }   //push opening bracket

  else if (expr[i] ==='>'){
                        // No matching opening bracket
    if (isEmpty()){
      balanced=0;
      break;
      }
      Pop();
      }
    }

// Stack must be empty for balanced expression.
  if (balanced && isEmpty()){
    Printf("Balanced Expression\n");}

  else
    {printf("unbalanced Expression\n");}

  return 0;
}
```

output:-

Enter expression : (A+B) * (C-D)

Balanced expression

③ Next Greater Element

Algorithm:-

① Traverse array from right to left.

② For each element, pop, stack elements that are <= current element.

③ If stack is empty, NGE = -1; else NGE = top of stack.

④ Push current element onto stack.

Code:-

```
# include <stdio.h>
# define MAX 100

int stack [MAX];

int top = -1;

void push (int val){
    stack [++top] = val; }

int pop(){
    return stack [top--]; }

int isEmpty(){
    return top == -1; }

int peek(){
    return stack [top]; }
```

```
int main() {
    int arr[MAX], nge[MAX], n;
printf("Enter number of elements");
scanf("%d", &n);
printf("Enter elements: ");
for (int i=0 ; i<n; i++) {
    scanf("%d", &arr[i]); }
// Traverse from right to left.
  for (int i=n-1; i>=0 ; i--) {
    //Pop elements smaller or equal to current.
    while (!isEmpty() && Peek() <= arr[i]) {
      pop();
      // if stack empty, no greater element exist.
      nge[i] = isEmpty() ?-1 : Peek();
      push(arr[i]);        // push current element
//print result
printf("\n element -> Next greater element\n");
for (int i=0 ; i<n; i++) {
    printf("%d -> %d \n", arr[i], nge[i]);
return 0; }
```

output:-

Enter number of elements: 5

Enter elements: 4 5 2 10 8

Element → Next greater element

4 → 5

5 → 10

2 → 10

10 → -1

8 → -1

④ Print queue simulation.

Algorithm:-

① Intialize front=-1 ; rear=-1 ; size=0

② Add document (Enque):

ⓐ If queue is full, Print "Queue full"

ⓑ Else if empty, set front= rear=0

ⓒ Else increment rear.

ⓓ Store document name at queue[rear], increment size.

③ Print document (Dequeue):

ⓐ If queue is empty, Print "No pending documents"

ⓑ Else Print document at que [front]

ⓒ If front ==rear, reset front = rear = -1

ⓓ Else increment front, decrement size.

④ DISPlay Pending:

ⓐ If empty, Print "Queue is empty".

ⓑ Else Print all documents from front to rear.

Program:-

```
#include <stdio.h>
#include <string.h>
// Maximum number of documents in the queue.
#define MAX 10
#define DOC_LEN 50
// Queue structure for printer.
char queue[MAX][DOC_LEN];
int front=-1, rear=-1, size=0;
// Check if queue is full.
int isFull(){
    return size == MAX;}
// Check if queue is empty.
int isEmpty(){
    return size == 0;}
// Add document to the queue.
void addDocument(char docName[]){
    if (isFull()){
        Printf ("Printer queue is full! Cannot add %s\n",
            docName);
    return;}
```

```
if (isEmpty()){
    front = rear = 0; }
else{
    rear = (rear+1) % MAX; }

strcpy (queue [rear] , docname);
size++;
printf ("Document %s' added to the printer queue.\n", docname);}

// print (remove) the front document.
void PrintDocument(){
    if (isEmpty()){
        Printf ("No Pending documents in the queue.\n");
        return;
        }
    Point("printing document: %s' \n", queue[front]);
    if (front == rear){
        front=rear =-1;}
    else {
        front = (front +1) % MAX;}
    size--;
    }

// display all pending documents
void displayQueue(){
    if (isEmpty()){
```

```
printf("No Pending documents. Queue is empty.\n");
return;}
printf("\n--- Pending Documents (%d) ---\n", size);
int i = front, count=0;
while(count <size){
  printf("%d. %s\n", count +1, queue[i]);
  i=(i+1) % MAX;
  count ++;}
printf("----------\n");}

int main(){
  int choice;
  char docname [DOC_LEN];
  printf("=====Printer queue simulation====\n");

do{ printf("\n1. Add Document \n");
  printf("2. Print Document \n");
  printf("3. Display Pending Documents \n");
  printf("4. Exit \n");
  printf("Enter choice : ");
  scanf("%d", &choice: ");
  getchar();

Switch (choice){
  Case 1:
      Printf("Enter document name: ");
```

```
fgets(docName, DOC_LEN, stdin);
docName[strcspn(docName, "\n")] = '\0';
addDocument(docName);
break;
Case 2:
    printDocument();
    break;
case 3:
    displayQueue();
    break;
Case 4:
    printf("Existing Printer queue. Goodbye!\n");
    break;
default:
    Printf("Invalid Choice. Try again.\n"); }
} while (choice != 4);
return 0; }
```

<u>output:-</u>

```
====Printer Queue Simulation====
1. Add Document
2. Print Document
3. Display Pending Documents.
4. Exit.
Enter choice: 1
Enter document name: Report.Pdf
Document 'Report.Pdf' added to the Printer queue.
```

(5) Circular queue Basic operations.

(1) Initialize front = -1, rear = -1

(2) Enqueue(value):

(a) If full ((rear+1) % MAX == front), print "queue Full"

(b) Else if empty, set front = rear = 0

(c) Else rear = (rear + 1) % MAX

(d) queue [rear] = value

(3) Dequeue:

(a) If empty (front == -1), print "queue Empty"

(b) store queue [front] as removed value.

(c) If front == rear (single element), reset front = rear = -1

(d) Else front = (front + 1) % MAX.

(4) Peek:

(a) If empty, print "queue Empty"

(b) Else print queue [front]

(5) Display:

(a) If empty, print "queue Empty"

(b) Else traverse from front to rear using modulo print each

Program:-

```
#include <stdio.h>
#define MAX 6
int queue[MAX];
int front = -1, rear = -1;
int isEmpty() {
        return (front == -1); }

//insert element into circular queue.
void enqueue (int value) {
    if (isFull()) {
        printf ("circular queue is Full! Cannot insert %d \n",
                                                value);
        return; }

    if (isEmpty()) {
        front = rear = 0; }
    else {
        rear = (rear + 1) % MAX; }
    queue [rear] = value;
    Printf ("Enqued : %d \n", value); }

//remove element from circular queque.
void dequeue() {
    if (isEmpty()) {
    Printf ("circular Queue is Empty! Nothing to remove. \n");
    return; }
    Printf ("Dequeued : %d \n", queue [front]);
    if (front == rear) {
        front = rear = -1; }
```

```
else {
    front = (front + 1) % MAX; }}

// Display front element without removing.
void peek(){
    if (isEmpty()){
        printf("Circular Queue is Empty \No front element\n");
    return;}
    printf("Front element (peek): %d \n", queue[front]); }

// Display all element in the circular queue.
void display(){
    if (isEmpty()){
        printf("circular Queue is Empty \n");
    return; }
    printf("\n --- circular Queue Elements --- \n");
    printf("Front ->");
    int i = front;
    while (1) {
        printf("%d", queue[i]);
        if (i == rear) break;
        i = (i+1) % MAX; }
    printf("<- Rear \n");
    printf("------------\n"); }
```

```
int main(){
    int choice, value;
    printf("=== Circular Queue operations ===\n");

do { printf("\n1. Enqueue\n");
    printf("\n2. Deque\n");
    printf("3. Peek\n");
    printf("4. display\n");
    printf("5. Exit\n");
    printf("Enter choice : ");
    scanf("%d", &choice);

Switch (choice) {

    Case 1:
        printf("Enter value to insert: ");
        scanf("%d", &value);
        enque (value);
        break;

    Case 2:
        dequeue();
        break;

    Case 3:
        peek();
        break;

    Case 4:
        display();
        break;

default : printf("Invalid choice, Try again.\n"); }

while (choice != 5);    return 0; }
```

Sample output:-

=== Circular Queue operations ===

1. Enqueue 2. Dequeue 3. Peek 4. Display 5. Exit

Enter choice : 1

Enter value to insert : 10

Enqueued : 10

Enter choice : 1

Enter value to insert : 30

Enqueued : 30

Enter choice : 4

--- circular queue elements ---

Front → 10 30 ← Rear.